

CSE411 — COMPUTER VISION

Project Task 6

Implementation — Part 3

Driver Drowsiness Detection System Using Computer Vision

Project detail

Information

Team Number

25

Team Members

Vangara Sridhar — 2023BCS0106

Suragouni Sahithi — 2023BCS0070

Vatte Ramya Sri — 2023BCS0208

Abhay Devanand Sawale — 2023BCS0046

Continuation of

Project Task 5 — Implementation, Part 2

Milestone

Approximately 75% of proposed project work

This report continues the earlier Task 1–5 submissions. It documents the current working prototype, the Task 6 computer-vision modules, their intended integration, current evidence, limitations, and the remaining 25%. The source

codebase is intentionally unchanged for this report.

1. Introduction and Task 6 Objective

Driver drowsiness is a gradual condition that cannot always be represented reliably by one isolated image. The project

therefore combines image classification with a temporal decision rule. The previous milestone connected the camera, FastAPI

backend, trained YOLO classifier, frontend result display, and five-second alarm logic into one runnable prototype.

Project Task 6 requires the team to complete approximately 75% of the proposed work, integrate completed modules into a

working prototype, demonstrate outputs, and state the unfinished 25%. The next step is to strengthen the computer-vision

pipeline with interpretable classical features while preserving the current application flow.

Area

Task 5 position

Task 6 position

Input

Webcam frame captured in the browser and uploaded as an image.

Keep the same capture path and use each frame for spatial and temporal CV analysis.

Preprocessing

OpenCV decode and BGR-to-RGB conversion.

Add HOG, Sobel gradients, histogram thresholding, and morphology as report-level CV components.

Temporal

reasoning

Five-second continuous Drowsy timer in frontend.

Add optical-flow magnitude between consecutive frames and retain the alarm timer.

Model

YOLO classification checkpoint with drowsiness classes.

Keep YOLO as the main classifier; expose CV metadata beside prediction probabilities.

Validation

Health check, prediction smoke test, lint, and build evidence.

Add feature-specific checks, latency

observations, and robustness

scenarios.

1.1 Scope boundary

Because the request explicitly says not to change the codebase, this PDF includes the required Task 6 module code as a

documented implementation appendix. The snippets are marked proposed and are not claimed as files already present in the

repository. This keeps the report technically honest while giving the team the code needed for the next controlled implementation step.

The completed system remains an assistive prototype and must not be treated as a replacement for safe driving, professional

medical assessment, or a certified vehicle safety system.

2. Existing Integrated System

The current project is organized as a frontend, backend, and machine-learning pipeline. A browser camera captures a frame,

the frontend sends it through the API client, FastAPI validates and decodes the upload, the inference service calls the loaded

YOLO model, and the response is rendered in the detection result interface. The frontend also counts session frames and

tracks continuous drowsiness for the alarm.

Stage

Current component

Current output

Camera capture

frontend/components/Camera.jsx

JPEG frame from the browser camera.

API request

frontend/lib/api.js

Multipart upload with timeout and

readable errors.

Validation/decode

backend/app/routes/predict.py +

image_processing.py

Accepted image decoded into an

OpenCV array.

Inference

backend/app/models/model.py +

services/inference.py

Class label, confidence, and all class

probabilities.

Feedback

frontend/app/page.jsx + DetectionResult.jsx

Status card, confidence, session

statistics, and streak.

Alarm

Frontend temporal timer and optional

alarm.mp3

Alarm after five seconds of continuous

Drowsy results.

2.1 Task 6 architecture

The proposed extension adds a classical computer-vision branch between image decoding and the final response. HOG

describes local gradients, Sobel and morphology describe edges and cleaned regions, and optical flow compares the current

frame with the previous frame. These outputs are returned as diagnostics so the feature work is observable rather than

calculated and discarded.

Camera frame t

|

+--> OpenCV decode / grayscale

|

+--> HOG descriptor

|

+--> Sobel magnitude -> Otsu threshold -> morphology

|

+--> frame t + previous frame t-1 -> Farneback optical flow

|

+--> YOLO classification -> label + confidence + probabilities

|

+--> JSON response + frontend streak/alarm

The YOLO prediction remains the primary learned decision. The classical branch supplies additional evidence and a stronger

syllabus connection; it should only influence the final alarm after testing and calibration.

3. Task 6 Computer-Vision Components

Three genuine computer-vision additions are documented for this milestone. Together they cover gradient-based descriptors,

motion analysis, edge detection, thresholding, segmentation, and morphological filtering.

Component

Purpose in this project

Output used by the system

HOG

Represent local gradient orientation patterns

around the driver or a selected face/eye region.

Feature length, mean, and

standard deviation in cv_features

metadata.

Optical flow

Compare consecutive monitoring frames to

estimate motion and stability.

Mean/max motion magnitude and

availability flag.

Sobel + thresholding

Extract horizontal/vertical gradients, combine them

into edge magnitude, and obtain a binary map using

Otsu thresholding.

Edge density and selected

threshold method.

Morphology

Remove small isolated regions and close small

gaps in the binary edge map.

Cleaned binary-region density

and morphology flag.

3.1 HOG feature extraction

Histogram of Oriented Gradients divides an image into local cells, estimates gradient directions, and aggregates them into

normalized blocks. In this project it can be applied to a fixed-size driver or face region. The feature summary is exposed in the

response so the team can verify that HOG is actually calculated and not silently ignored.

3.2 Optical flow for live monitoring

The frontend already produces a sequence of frames approximately one second apart during live monitoring. Farneback

optical flow can estimate the displacement field between consecutive grayscale images. A low value may indicate a stable

frame, while larger values can reflect head movement, camera movement, or background changes. It is therefore an

additional signal, not a direct drowsiness label.

The tracker must reset when monitoring stops, when the camera changes, or when a frame is missing. State should be scoped

to a monitoring session so one user's previous frame cannot contaminate another session.

4. Implementation Appendix — Proposed CV Module

The following code is included in the report as the Task 6 implementation design. The intended module is

backend/app/services/cv_features.py. It is not written into the codebase in accordance with the instruction to leave all source

files untouched.

A. HOG and Sobel/threshold/morphology — proposed cv_features.py

```
import cv2
```

```
import numpy as np
```

```
def compute_hog(image: np.ndarray) -> dict:
```

```
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```

gray = cv2.resize(gray, (128, 128))

descriptor = cv2.HOGDescriptor(
    _winSize=(128, 128), _blockSize=(16, 16),
    _blockStride=(8, 8), _cellSize=(8, 8), _nbins=9
)

values = descriptor.compute(gray)

return {"enabled": True,
        "feature_length": int(values.size),
        "mean": float(values.mean()),
        "std": float(values.std())}

def compute_sobel_features(image: np.ndarray) -> dict:
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    gx = cv2.Sobel(gray, cv2.CV_32F, 1, 0, ksize=3)
    gy = cv2.Sobel(gray, cv2.CV_32F, 0, 1, ksize=3)
    magnitude = cv2.magnitude(gx, gy)
    magnitude = cv2.normalize(magnitude, None, 0, 255, cv2.NORM_MINMAX)
    _, binary = cv2.threshold(magnitude.astype(np.uint8), 0, 255,
                              cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    kernel = np.ones((3, 3), np.uint8)
    opened = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
    cleaned = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, kernel)
    return {"edge_density": float(np.mean(cleaned > 0)),
            "threshold": "otsu", "morphology": True}

```

B. Optical-flow tracker — proposed cv_features.py

```

def compute_optical_flow(previous_gray, frame: np.ndarray):
    current_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    if previous_gray is None:
        return current_gray, {"available": False, "mean_magnitude": 0.0}
    flow = cv2.calcOpticalFlowFarneback(
        previous_gray, current_gray, None, 0.5, 3, 15, 3, 5, 1.2, 0
    )

```

```
)

magnitude, _ = cv2.cartToPolar(flow[..., 0], flow[..., 1])

return current_gray, {"available": True,

"mean_magnitude": float(np.mean(magnitude)),

"max_magnitude": float(np.max(magnitude))}
```

5. Inference and API Integration Design

The existing inference function already returns a structured prediction containing the label, confidence, and class probabilities.

The Task 6 extension should attach a `cv_features` object to that same prediction instead of changing the existing response

contract. This keeps the frontend compatible while making the additional analysis available for demonstration and future

evaluation.

C. Proposed inference metadata integration

After the existing YOLO prediction has been created:

```
cv_features = {

"hog": compute_hog(image),

"sobel": compute_sobel_features(image),

"optical_flow": flow_info,

}

prediction["cv_features"] = cv_features

return prediction

{

"success": true,

"prediction": {

"label": "Drowsy",

"confidence": 0.91,

"status": "drowsy",

"all_probs": {"Alert": 0.03, "Drowsy": 0.91,

"Low_Vigilant": 0.04, "Non_Drowsy": 0.02},

"cv_features": {
```

```
"hog": {"enabled": true, "feature_length": 8100},  
"optical_flow": {"available": true, "mean_magnitude": 0.18},  
"sobel": {"edge_density": 0.21, "threshold": "otsu",  
"morphology": true}  
}  
}  
}
```

5.1 Existing temporal alarm connection

The current frontend starts timing when the prediction status becomes drowsy and resets the timer when the result is no

longer drowsy. After the configured five-second duration, the alarm begins. This is important for the project requirement: one

still image with closed eyes cannot prove five seconds of continuous closure; a live sequence is required.

Condition

System response

Drowsy prediction begins

Store the start time and increase the displayed drowsy streak.

Drowsy persists for five seconds

Start the optional alarm.mp3 or browser tone fallback and show visual

alarm feedback.

Alert / Non-Drowsy result

Reset the streak and stop the alarm.

Monitoring stopped

Clear alarm state and reset temporal / optical-flow session state.

6. Implementation Evidence and Results

The Task 5 report recorded the working baseline that Task 6 builds on. These results are repository-grounded and are

separated from the proposed Task 6 feature code because the source files were not modified for this submission.

Test / check

Observed result

Interpretation

GET /health

healthy; model_loaded = true

The backend starts and the configured
model is available.

POST /predict smoke test

Drowsy; confidence about 0.91;
severity 2

Image decode, inference, and response
formatting work end to end.

Model inspection

Alert, Drowsy, Low_Vigilant,
Non_Drowsy

The active checkpoint is a drowsiness
classifier.

Frontend lint

Passed

No ESLint warnings or errors were reported.

Frontend build

Passed

Next.js production compilation succeeds.

Alarm threshold

Five seconds of continuous Drowsy
status

Temporal logic is present and requires live
consecutive results.

6.1 Task 6 validation matrix

Feature

Validation procedure

Expected evidence

HOG

Run a valid frame through the feature function.

Non-zero descriptor length

and summary statistics.

Sobel / morphology

Run gradient magnitude, Otsu threshold, opening, and closing.

Edge density and morphology

flag in metadata.

Optical flow

Send moving consecutive frames and identical consecutive frames.

Movement produces a higher magnitude than identical frames.

Alarm

Run continuous live monitoring with Drowsy outputs for at least five seconds.

Audio/tone and visual alarm appear after the threshold.

Reset

Stop monitoring or send a non-drowsy result.

Alarm, streak, and previous-frame state reset.

The feature values used in a final demonstration must be recorded from the running implementation. Example values in the

code appendix are response-format examples, not measured results from the untouched repository.

7. Limitations, Safety, and Reliability

The Task 6 components improve the breadth of the computer-vision pipeline, but they do not eliminate the limitations

identified in the earlier reports. A careful report must distinguish a working prototype from a validated driver-safety product.

Limitation

Why it matters

Required follow-up

Frame-level classifier

A single prediction can be unstable and does not directly measure duration of eye closure.

Use temporal smoothing and add facial landmarks or a dedicated eye-state detector.

HOG sensitivity

Lighting, glasses, pose, and image quality change gradient distributions.

Test across subjects, lighting, glasses, and camera positions.

Optical-flow ambiguity

Motion can come from the head, camera, background, or compression rather than drowsiness.

Use a face/eye ROI, reset state correctly, and report flow as supporting evidence.

Audio permissions

Browsers may block audio until the user interacts with the page.

Prime audio on Start Monitoring and test alarm.mp3 in the target browser.

Dataset generalization

Random frame splits can overestimate performance when frames share subjects or

videos.

Use subject/video-level held-out

evaluation.

7.1 What counts as a successful Task 6 demonstration

The team should show the working frontend, a backend response containing the CV metadata, a consecutive-frame

optical-flow comparison, and the five-second alarm behavior. The demonstration should also state that HOG, optical flow,

Sobel, thresholding, and morphology are documented here but were not applied to the repository source because the

submission instruction required the codebase to remain untouched.

The alarm should be described as a warning aid. It should never encourage a driver to continue driving while drowsy; the safe

action is to stop and rest.

8. Remaining 25% of the Project

The final 25% is the validation and reliability phase. It should convert the integrated prototype into a defensible final project by

measuring performance, validating real-world conditions, and completing the deployment and documentation work.

Priority

Remaining work

Completion evidence

High

Formal model evaluation on a subject/video-level held-out set:

accuracy, precision, recall, F1, confusion matrix, and per-class results.

Reproducible evaluation

script and metric tables.

High

Direct eye-closure measurement using facial landmarks or a dedicated eye-state model; measure continuous closure duration.

Eye-region evidence tied

to the alarm decision.

High

Temporal robustness: smoothing, missing-frame handling,
configurable thresholds, false-alarm and missed-alarm analysis.

Scenario matrix across
people and conditions.

Medium

Production validation: environment variables, CORS, model
availability, cold starts, timeout, and frontend-backend connectivity.

Production smoke test
and deployment checklist.

Medium

Alarm and safety UX: alarm.mp3 testing, audio permission handling,
mute/reset control, visible countdown, and safety messaging.

Final live demo and user
instructions.

Low

Final documentation, session export, experiment tracking, model
provenance, and reproducibility notes.

Final README and
submission package.

Recommended order

First complete the held-out evaluation and direct eye-closure measurement. Then tune temporal behavior and validate the

deployment environment. Finally polish the alarm experience and documentation. HOG and optical flow
strengthen the

computer-vision coursework component, but they should not be presented as proof of eye closure by themselves.

9. Conclusion

Project Task 6 continues the Driver Drowsiness Detection System from the functioning Task 5 web prototype
toward a

broader and more interpretable computer-vision pipeline. The existing camera capture, validation, YOLO
inference, frontend

feedback, session statistics, and five-second warning mechanism remain the system foundation.

The Task 6 design contributes three connected CV components: HOG for local gradient structure, optical flow for frame-to-frame motion, and Sobel thresholding with morphology for edge and binary-region analysis. The response design

exposes these measurements so they can be inspected and evaluated rather than discarded.

The source codebase was not changed. The required module and integration snippets are included in this report as a

controlled implementation plan. The final 25% is the work needed to implement and validate the feature branch, measure

generalization, add direct eye-state evidence, verify deployment, and finalize safety-oriented documentation.

Code and project files referenced

backend/app/services/image_processing.py · backend/app/services/inference.py · backend/app/routes/predict.py · backend/app/models/model.py

· frontend/app/page.jsx · frontend/components/Camera.jsx · ml_pipeline/train.py · backend/weights/best.pt

Submission note

This report is prepared as a continuation of the earlier Task 5 report and follows the Project Task 6 requirements: project title, team number, all

team members, approximately 75% milestone, integrated-system discussion, current evidence, and remaining 25%.